



Cloud-Native SIG and CAKE present...

Pocket-Sized Kubernetes: Building and Deployment with Raspberry Pi Clusters

16 June 2026, RH007, Durham University



github.com/cloud-native-sig/hpcdays26-pocket-sized-kubernetes
cloud-native-sig.github.io/hpcdays26-pocket-sized-kubernetes



Agenda

1. Introductions
2. Kubernetes Architecture & Cluster Design
3. Section 1—Building your cluster
4. Lunch break
5. Kubernetes Deployments & Namespaces
6. Section 2—Using your cluster
7. Summary, Q&A, Follow-up opportunities

Introductions



Who We Are

Tutorial from the **Cloud-Native Special Interest Group** with support from the **Computational Abilities Knowledge Exchange** partnership.

New community of RSEs/dRTPs for cloud-native technologies in research software and digital infrastructure.

Get involved at <https://cloudnative-sig.ac.uk/>

Lewis Sampson (DAFNI), lewis.sampson@stfc.ac.uk

Piper Fowler-Wright (Rosalind Franklin Institute), piper.fowler-wright@rfi.ac.uk



Today's Tutorial

By the end you will have:

- An understanding of Kubernetes architecture and its use in research computing
- Built a multi-node K3s cluster on Raspberry Pi hardware
- Deployed basic applications and observed key Kubernetes features
- Gained practical skills transferable to HPC and cloud environments

Everything you learn today can be used in your own work.

Following Along at Home

Code and resources available on the [GitHub repo](#). If you want to recreate with your own hardware, see the [extra reading section](#) of the GitHub pages.

Introduction to Kubernetes



What is Kubernetes (k8s)

Kubernetes is an open-source container orchestration platform to automate the deployment, scaling & management of containerised applications across groups of machines.



Kubernetes vs Docker Compose

- **Docker:** run apps from built images in containers (sandboxed environments)
- **Docker Compose:** groups of containers with networking and storage on a single host
- **Kubernetes:** Orchestration pipeline for multi-host, multi-container applications.



Kubernetes vs Docker Compose

- **Docker:** run apps from built images in containers (sandboxed environments)
- **Docker Compose:** groups of containers with networking and storage on a single host
- **Kubernetes:** Orchestration pipeline for multi-host, multi-container applications.

K8s allows complex containerised applications to run across multiple hosts in a cluster with powerful automation and management features.



Kubernetes in Research Computing

Use if need to manage multiple containerised services across different machines. Or may benefit from

- Automatic workload scaling based on demand
- Rolling updates with minimal downtime
- Self-healing resilient systems with high-availability
- Portability between on-prem and cloud infrastructure



Kubernetes in Research Computing

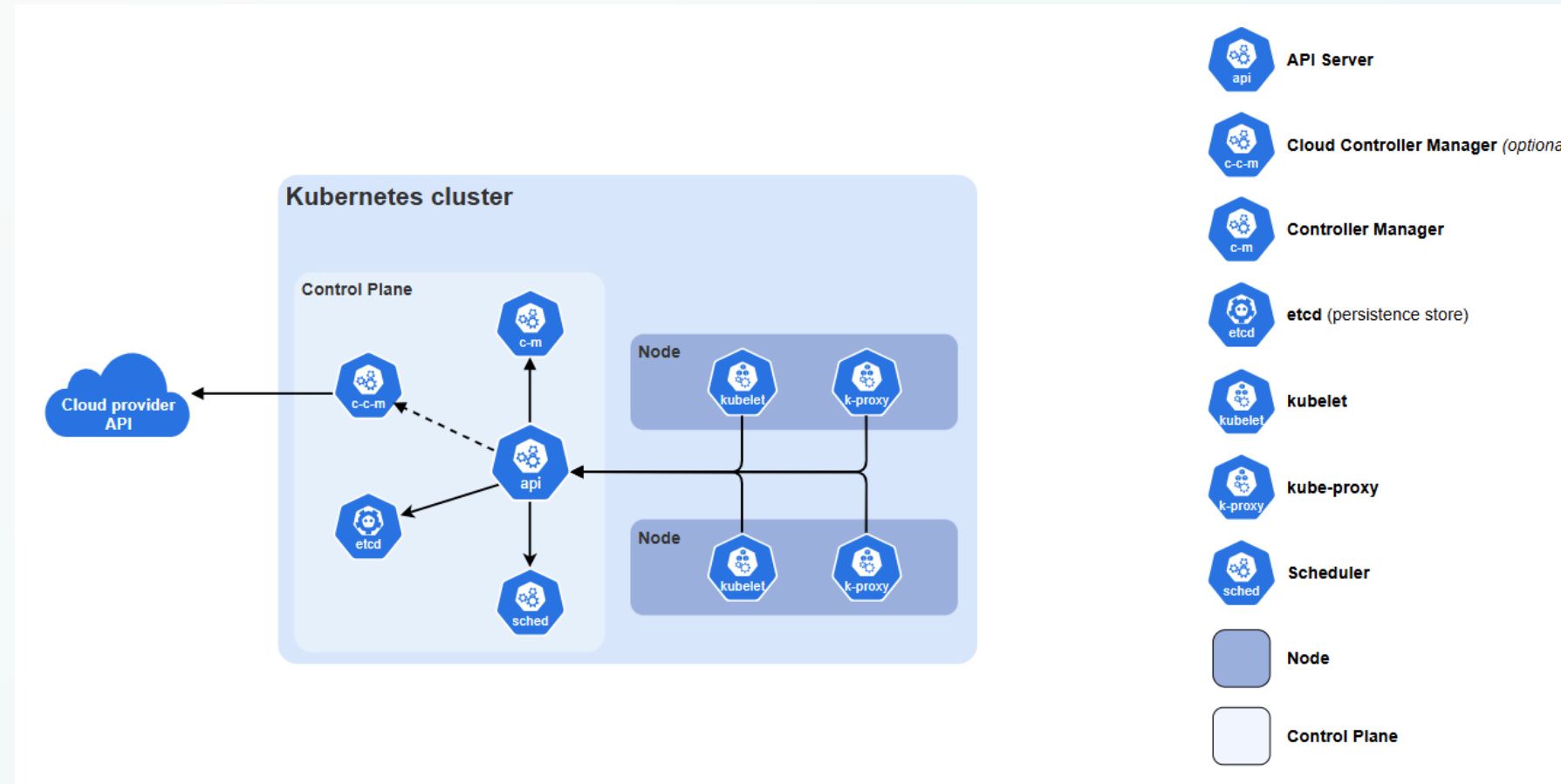
Adds complexity and not typically suitable for:

- Simple container applications on a single host (use Docker Compose)
- HPC job management (use SLURM)
- Large-scale parallel filesystems (e.g., Lustre)
- Services offered by a cloud provider or technology you are already invested in

For more see the [extra reading section](#) on Kubernetes and HPC



Architecture Overview



Components of a Kubernetes cluster (kubernetes.io)



Key Components

- **Node:** Physical or virtual machine
- **Control Plane (Node):** Brains of cluster, acts based on current cluster state vs target state
- **Worker Node:** Run containerised applications using a container runtime.
- **Pod:** Smallest deployable unit—one or more containers sharing storage/network
- **Deployment:** Target state of identical pods serving an app
- **Service:** Network endpoint for pods

Other critical components include the Controller Manager and Scheduler, and etcd (key-value store for cluster data).



How Kubernetes Works

Declarative: define target state of applications—Kubernetes works to achieve it

Example: On *node failure* Kubernetes distributes workload to other nodes



How Kubernetes Works

Declarative: define target state of applications—Kubernetes works to achieve it

Example: On *node failure* Kubernetes distributes workload to other nodes

Reconciliation model



How Kubernetes Works

Declarative: define target state of applications—Kubernetes works to achieve it

Example: On *node failure* Kubernetes distributes workload to other nodes

Reconciliation model

GitOps: Infrastructure as code (`.yaml`), workflow managers, e.g., [ArgoCD](#)

Designing a Cluster



Kubernetes Distributions

- Vanilla k8s, i.e., [kubeadm](#) (CNCF)—industry standard, fully-featured
- [RKE2](#) (Rancher Kubernetes Engine 2)—enterprise, security & compliance
- [K3s](#) (Rancher)—Minimal resources & installation, e.g., IoT / Edge Computing
- [MicroK8s](#) (Canonical)—Batteries included, lightweight
- [Minikube](#)—Single-node cluster for local development



K3s and Hardware Requirements

K3s for our RPi clusters: minimal footprint & ease of installation

Control node—Min. 2 Cores/2G RAM

Worker node—Min. 1 Core/512M RAM

In K3s, the node running the control-plane is referred to as the **Server**, and all other nodes **Agents**.



Cluster Hardware

- 4x 4G RPi model 4B (or 400)
- RPi OS Lite (Debian-based, non-GUI)
- Select binaries and custom container images for air-gapped installations (see `workshop-setup/` on GitHub)

Exercise 1—Connecting to Nodes



Network Information

Each table should have a note with:

- RPi IP addresses
- SSH login credentials
- Router WiFi credentials

Connect your laptop to Router `TP-Link_AP_2A5A_01`

Our router does *not* provide internet access. Consider installing [kubectI](#) locally first.



Verify Connectivity

Once connected to the router, as a group check access to each node:

```
ssh chef@192.168.x.xxx  
hostname  
exit
```

And that the worker nodes are reachable from the control:

```
ssh chef@192.168.x.xxx  
ping -c3 192.168.x.yyy
```



Troubleshooting

If a node is unreachable:

1. Check powered on
2. Check IP address
3. Check `sshd` is running

If needed, connect RPi to an external display to troubleshoot & configure IP using `nmcli/nmtui`.



Optional: Configure Host Aliases

Edit `/etc/hosts` to use `ssh <user>@kmaster` etc. instead of IP addresses:

```
192.168.x.xxx    kmaster
192.168.x.yyy    kworker1
```

Or configure SSH aliases in `~/.ssh/config`:

```
Host kworker01
  HostName 192.168.x.yyy
  User chef
```

Then `ssh kworker01` would connect to the node. You can also setup SSH keys for quicker logins (e.g., `ssh-keygen -t ed25519; ssh-copy-id chef@kmaster`)

Exercise 2—Installing K3s & Creating the Cluster



Strategy

1. Install K3s on Control
2. Retrieve Join Token for Workers
3. Install K3s on Workers and Join to Cluster

Split into ~2 per node (pair program on one device)

Explore the [K3S Documentation](#) and ask questions!



Control Node Installation

On `kmaster` (code on GitHub pages/slides):



Control Node Verification

Check K3s is running:

Check node status:

Which version of K3s was installed?



Worker Node Join Token

Everyone on a worker node will need the following from the *control*:

Copy this to, e.g., `/home/chef/node-token` on the Worker and assign



Worker Installation

With `CONTROL_NODE` and `CONTROL_TOKEN` set (code on GitHub/slides):



Worker Verification

As on the master, check `sudo kubectl get nodes` . All nodes should have `STATUS: ready` but no role (see [snippet on GitHub pages](#) to fix this).

Optional: kubectl without sudo

It is convenient to set

to use `kubectl` without `sudo` .

Exercise 3—Using kubectl



Kubectl Syntax

Command-line interface for Kubernetes:

- `<command>` —action
- `<type>` —Kubernetes resource (nodes, pods, deployments etc.)
- `[name]` —optional resource name
- `[flags]` —optional parameters



Optional: kubectl From Your Laptop (Requires kubectl!)

Copy K3s config from the control:

Edit `server` :

Set `KUBECONFIG` :



About: Namespaces

Separate groups of cluster resources:

A large, empty rectangular box with a light blue border, intended for a diagram or text explaining how namespaces separate cluster resources.

Can filter resources by namespace, e.g.,

A large, empty rectangular box with a light blue border, intended for a diagram or text explaining how to filter resources by namespace.

Change default namespace from `default` (empty) to `kube-system` :



Exploring Resources (kube-system)

Get resource (`pods` , `deployments` , `services` ...):

`get all` may be used to list everything within a namespace

Chose a pod to inspect further:



Accessing Containers

`exec` can be used to run commands in a container (like docker) within a pod:

See examples in the next session



About: Storage

Pods are *ephemeral*. Storing data requires

- Persistent Volumes (PV) and Persistent Volume Claims (PVC), OR
- `ConfigMaps` for configuration data, e.g., environmental variables, CLI arguments

Examples of PV/PVC in next session; for `ConfigMaps` see [Updating with ConfigMaps](#)



Section 1—Summary

So far:

- Introduced Kubernetes architecture (Declarative) and design (Control/Workers, Distributions)
- Built RPi Cluster with K3s
- Setup `kubect1` to inspect Kubernetes resources

Next: Using your cluster!

Section 2—Using Your Cluster



Prep: Get Demo Deployments

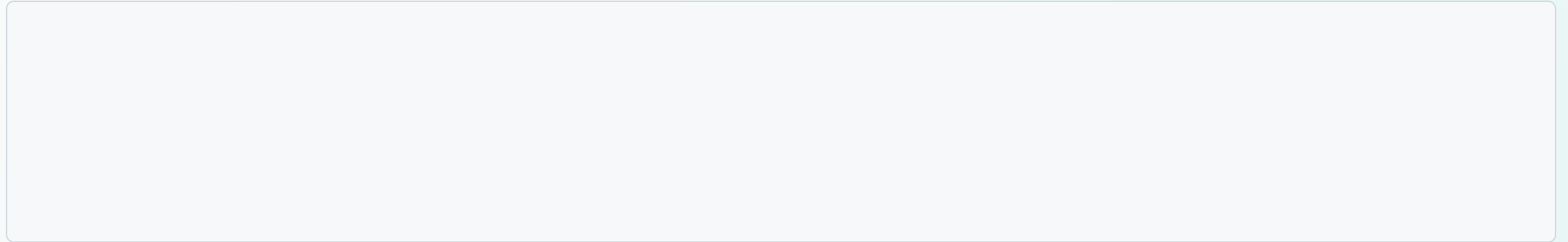
Clone repository to your laptop if you haven't already:

Reconnect to `TP-Link_AP_2A5A_01` and copy `resources` to your *node*:



About: Deployments

- Defines target *state* of your app (which container images, how many replica pods to run them)
- Kubernetes works to reconcile actual state with this target
- Deployed from a `manifest.yaml` using `apply` :



Kubernetes handles the rest!

`kubectl rollout restart deployment/deployment-name` can be used to initiate a *rolling restart* of a deployment if needed



Exercises: Over to You!



Section 2 — Using your cluster

Introduction

Exercise 1 — Services, networking and Monitoring

1a) Services and networking

1b) Monitoring and Telemetry

Exercise 2 — Resource Management and Failure Recovery

Exercise 3 — Persistent Storage

Exercise 4 — Jobs and Batch Execution

Tips

- Can be done in any order
- Work on *different* exercises in pairs
- Use namespaces and cleanup resources (see Exercises)



Section 2—Summary

- Application *deployments* are specified by manifest files
- Services provide networking endpoints
- Examples with networking & monitoring, resource management & recovery, storage & jobs

Feel free to continue working on the exercises at home (e.g., using Minikube)



Knowledge Exchange opportunities

Cloud_Native SIG

This workshop was brought to you by the Cloud-Native SIG and CAKE, with support from the Software Sustainability Institute

Join us:

-  cloudnative-sig@jiscmail.ac.uk
-  cloudnative-sig.ac.uk



Knowledge Exchange Opportunities

CAKE Fellowship

Read more: <https://www.cake.ac.uk/ke-fellowships/cohort1>

SCD Kubernetes collaboration group

Contact Lewis lewis.sampson@stfc.ac.uk for more information.

Thank you for your participation